

04_exercise_run4_0.9984Accuracy

September 29, 2025

```
[36]: from pathlib import Path
import numpy as np
import torch
from typing import List
from torch.nn.utils.rnn import pad_sequence
from mltrainer import rnn_models, Trainer
from torch import optim

from mads_datasets import datatools
import mltrainer
mltrainer.__version__
```

```
[36]: '0.2.4'
```

1 1 Iterators

We will be using an interesting dataset. [link](#)

From the site: > The SmartWatch Gestures Dataset has been collected to evaluate several gesture recognition algorithms for interacting with mobile applications using arm gestures. Eight different users performed twenty repetitions of twenty different gestures, for a total of 3200 sequences. Each sequence contains acceleration data from the 3-axis accelerometer of a first generation Sony SmartWatch™, as well as timestamps from the different clock sources available on an Android device. The smartwatch was worn on the user's right wrist.

```
[37]: from mads_datasets import DatasetFactoryProvider, DatasetType
from mltrainer.preprocessors import PaddedPreprocessor
preprocessor = PaddedPreprocessor()

gesturesdatasetfactory = DatasetFactoryProvider.create_factory(DatasetType.
↳GESTURES)
streamers = gesturesdatasetfactory.create_datastreamer(batchsize=32,↳
↳preprocessor=preprocessor)
train = streamers["train"]
valid = streamers["valid"]
```

2025-09-29 11:13:32.877 | INFO |

```
mads_datasets.base:download_data:121 - Folder
```

```
already exists at /Users/nickreinders/.cache/mads_datasets/gestures
```

```
100%|      | 2600/2600 [00:00<00:00, 5770.14it/s]
```

```
100%|      | 651/651 [00:00<00:00, 5908.95it/s]
```

```
[38]: len(train), len(valid)
```

```
[38]: (81, 20)
```

```
[39]: trainstreamer = train.stream()
      validstreamer = valid.stream()
      x, y = next(iter(trainstreamer))
      x.shape, y
```

```
[39]: (torch.Size([32, 32, 3]),
      tensor([ 9, 14,  1, 11, 12,  1,  7, 14,  0, 15,  8, 19,  1, 18, 13, 16, 13,  8,
              5,  7, 15,  1, 13,  4, 19, 14, 11, 18, 19, 11,  1, 11]))
```

Can you make sense of the shape? What does it mean that the shapes are sometimes (32, 27, 3), but a second time might look like (32, 30, 3)? In other words, the second (or first, if you insist on starting at 0) dimension changes. Why is that? How does the model handle this? Do you think this is already padded, or still has to be padded?

2 2 Excercises

Lets test a basemodel, and try to improve upon that.

Fill the gestures.gin file with relevant settings for `input_size`, `hidden_size`, `num_layers` and `horizon` (which, in our case, will be the number of classes...)

As a rule of thumbs: start lower than you expect to need!

```
[40]: from mltrainer import TrainerSettings, ReportTypes
      from mltrainer.metrics import Accuracy

      accuracy = Accuracy()
```

```
[41]: model = rnn_models.BaseRNN(
      input_size=3,
      hidden_size=64,
      num_layers=1,
      horizon=20,
      )
```

Test the model. What is the output shape you need? Remember, we are doing classification!

```
[42]: yhat = model(x)
      yhat.shape
```

```
[42]: torch.Size([32, 20])
```

Test the accuracy

```
[43]: accuracy(y, yhat)
```

```
[43]: 0.0625
```

What do you think of the accuracy? What would you expect from blind guessing?

Check shape of y and yhat

```
[44]: yhat.shape, y.shape
```

```
[44]: (torch.Size([32, 20]), torch.Size([32]))
```

And look at the output of yhat

```
[45]: yhat[0]
```

```
[45]: tensor([-0.2363, -0.0141, -0.1311, -0.0325, -0.0005, -0.0117, -0.1894,  0.0185,  
          -0.0265,  0.0040, -0.0652,  0.0124, -0.0196, -0.0590, -0.1651,  0.0098,  
          -0.0237, -0.1172,  0.1219,  0.0463], grad_fn=<SelectBackward0>)
```

Does this make sense to you? If you are unclear, go back to the classification problem with the MNIST, where we had 10 classes.

We have a classification problem, so we need Cross Entropy Loss. Remember, [this has a softmax built in](#)

```
[46]: loss_fn = torch.nn.CrossEntropyLoss()  
      loss = loss_fn(yhat, y)  
      loss
```

```
[46]: tensor(2.9938, grad_fn=<NllLossBackward0>)
```

```
[47]: import torch  
      if torch.backends.mps.is_available() and torch.backends.mps.is_built():  
          device = torch.device("mps")  
          print("Using MPS")  
      elif torch.cuda.is_available():  
          device = "cuda:0"  
          print("using cuda")  
      else:  
          device = "cpu"  
          print("using cpu")  
  
      # on my mac, at least for the BaseRNN model, mps does not speed up training  
      # probably because the overhead of copying the data to the GPU is too high  
      # so i override the device to cpu  
      device = "cpu"
```

```
# however, it might speed up training for larger models, with more parameters
```

Using MPS

Set up the settings for the trainer and the different types of logging you want

```
[48]: settings = TrainerSettings(  
    epochs=3, # increase this to about 100 for training  
    metrics=[accuracy],  
    logdir=Path("gestures"),  
    train_steps=len(train),  
    valid_steps=len(valid),  
    reporttypes=[ReportTypes.TOML, ReportTypes.TENSORBOARD, ReportTypes.MLFLOW],  
    scheduler_kwargs={"factor": 0.5, "patience": 5},  
    earlystop_kwargs = {  
        "save": False, # save every best model, and restore the best one  
        "verbose": True,  
        "patience": 5, # number of epochs with no improvement after which  
        ↪ training will be stopped  
        "delta": 0.0, # minimum change to be considered an improvement  
    }  
)  
settings
```

```
[48]: epochs: 3  
metrics: [Accuracy]  
logdir: gestures  
train_steps: 81  
valid_steps: 20  
reporttypes: [<ReportTypes.TOML: 'TOML'>, <ReportTypes.TENSORBOARD:  
'TENSORBOARD'>, <ReportTypes.MLFLOW: 'MLFLOW'>]  
optimizer_kwargs: {'lr': 0.001, 'weight_decay': 1e-05}  
scheduler_kwargs: {'factor': 0.5, 'patience': 5}  
earlystop_kwargs: {'save': False, 'verbose': True, 'patience': 5, 'delta': 0.0}
```

```
[49]: import torch.nn as nn  
import torch  
from torch import Tensor  
from dataclasses import dataclass  
  
@dataclass  
class ModelConfig:  
    input_size: int  
    hidden_size: int  
    num_layers: int  
    output_size: int  
    dropout: float = 0.0
```

```

class GRUModel(nn.Module):
    def __init__(
        self,
        config,
    ) -> None:
        super().__init__()
        self.config = config
        self.rnn = nn.GRU(
            input_size=config.input_size,
            hidden_size=config.hidden_size,
            dropout=config.dropout,
            batch_first=True,
            num_layers=config.num_layers,
        )
        self.linear = nn.Linear(config.hidden_size, config.output_size)

    def forward(self, x: Tensor) -> Tensor:
        x, _ = self.rnn(x)
        last_step = x[:, -1, :]
        yhat = self.linear(last_step)
        return yhat

```

```

[50]: config = ModelConfig(
    input_size=3,
    hidden_size=64,
    num_layers=1,
    output_size=20,
    dropout=0.0,
)

```

```

[51]: import mlflow
from datetime import datetime

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("gestures")
model_dir = Path("gestures").resolve()
if not model_dir.exists():
    model_dir.mkdir(parents=True)

with mlflow.start_run():
    mlflow.set_tag("model", "modelname-here")
    mlflow.set_tag("dev", "your-name-here")
    config = ModelConfig(
        input_size=3,
        hidden_size=64,
        num_layers=1,
        output_size=20,
    )

```

```

        dropout=0.1,
    )

    model = GRUModel(
        config=config,
    )

    trainer = Trainer(
        model=model,
        settings=settings,
        loss_fn=loss_fn,
        optimizer=optim.Adam,
        traindataloader=trainstreamer,
        validdataloader=validstreamer,
        scheduler=optim.lr_scheduler.ReduceLROnPlateau,
        device=device,
    )
    trainer.loop()

    if not settings.earlystop_kwargs["save"]:
        tag = datetime.now().strftime("%Y%m%d-%H%M-")
        modelpath = modeldir / (tag + "model.pt")
        torch.save(model, modelpath)

```

/Users/nickreinders/Documents/Master Applied Data Science deeltijd
HAN/Deployment/Raoul_repo/MADS-MachineLearning-course/.venv/lib/python3.11/site-
packages/torch/nn/modules/rnn.py:123: UserWarning: dropout option adds dropout
after all but last recurrent layer, so non-zero dropout expects num_layers
greater than 1, but got dropout=0.1 and num_layers=1

```

warnings.warn(
2025-09-29 11:13:33.673 | INFO      |
mltrainer.trainer:dir_add_timestamp:24 - Logging

```

to gestures/20250929-111333

```

2025-09-29 11:13:33.674 | INFO      |
mltrainer.trainer:__init__:68 - Found

```

earlystop_kwargs in settings.Set to None if you dont want earlystopping.

```

100%|      | 81/81 [00:00<00:00, 350.21it/s]
2025-09-29 11:13:33.932 | INFO      |
mltrainer.trainer:report:209 - Epoch 0 train

```

```

2.8672 test 2.4923 metric ['0.1031']
100%|      | 81/81 [00:00<00:00, 352.28it/s]
2025-09-29 11:13:34.186 | INFO      |
mltrainer.trainer:report:209 - Epoch 1 train

```

```

2.3410 test 2.2759 metric ['0.1594']
100%|      | 81/81 [00:00<00:00, 346.50it/s]

```

```
2025-09-29 11:13:34.445 | INFO      |
mltrainer.trainer:report:209 - Epoch 2 train
2.1789 test 2.0505 metric ['0.2578']
100%|      | 3/3 [00:00<00:00, 3.91it/s]
```

Try to update the code above by changing the hyperparameters.

To discern between the changes, also modify the tag `mlflow.set_tag("model", "new-tag-here")` where you add a new tag of your choice. This way you can keep the models apart.

```
[52]: trainer.loop() # if you want to pick up training, loop will continue from the
      ↪ last epoch
```

```
100%|      | 81/81 [00:00<00:00, 341.65it/s]
2025-09-29 11:13:34.710 | INFO      |
mltrainer.trainer:report:175 - Resuming epochs
from previous training at 3
2025-09-29 11:13:34.718 | INFO      |
mltrainer.trainer:report:209 - Epoch 3 train
1.9530 test 1.8231 metric ['0.3406']
100%|      | 81/81 [00:00<00:00, 326.88it/s]
2025-09-29 11:13:34.991 | INFO      |
mltrainer.trainer:report:209 - Epoch 4 train
1.7028 test 1.5916 metric ['0.3766']
100%|      | 81/81 [00:00<00:00, 356.73it/s]
2025-09-29 11:13:35.243 | INFO      |
mltrainer.trainer:report:209 - Epoch 5 train
1.5017 test 1.4265 metric ['0.4828']
100%|      | 3/3 [00:00<00:00, 3.79it/s]
```

```
[53]: mlflow.end_run()
```

Exercises:

- try to improve the RNN model
- test different things. What works? What does not?
- experiment with either GRU or LSTM layers, create your own models. Have a look at `mltrainer.rnn_models` for inspiration.
- experiment with adding Conv1D layers. Think about the necessary input-output dimensions of your tensors before and after each layer.

You should be able to get above 90% accuracy with the dataset. Create a report of 1 a4 about your experiments.

vanaf hier dingen proberen

```
[54]: class LSTMmodel(nn.Module):
      def __init__(self, config) -> None:
```

```

    super().__init__()
    self.config = config
    self.rnn = nn.LSTM(
        input_size=config.input_size,
        hidden_size=config.hidden_size,
        dropout=config.dropout,
        batch_first=True,
        num_layers=config.num_layers,
    )
    self.linear = nn.Linear(config.hidden_size, config.output_size)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x, _ = self.rnn(x)
        last_step = x[:, -1, :]      # neem laatste timestep
        yhat = self.linear(last_step)
        return yhat

```

```

[55]: config = ModelConfig(
    input_size=3,      # 3 sensorkanalen (accelerometer)
    hidden_size=128,   # groter dan 64 proberen
    num_layers=2,      # meer lagen
    output_size=20,    # 20 klassen (gestures)
    dropout=0.2,       # beetje regularisatie
)

```

```

[56]: with mlflow.start_run():
    mlflow.set_tag("model", "LSTM-128x2")
    mlflow.set_tag("dev", "nick")

    model = LSTMModel(config=config)

    trainer = Trainer(
        model=model,
        settings=settings,
        loss_fn=loss_fn,
        optimizer=optim.Adam,
        traindata_loader=trainstreamer,
        validdata_loader=validstreamer,
        scheduler=optim.lr_scheduler.ReduceLROnPlateau,
        device=device,
    )
    trainer.loop()

```

```

2025-09-29 11:13:35.283 | INFO      |
mltrainer.trainer:dir_add_timestamp:24 - Logging
to gestures/20250929-111335
2025-09-29 11:13:35.284 | INFO      |

```



```

mltrainer.trainer:__init__:68 - Found
earlystop_kwargs in settings.Set to None if you dont want earlystopping.
0%|          | 0/3 [00:00<?, ?it/s]2025-09-29
11:13:35.284 | INFO          |
mltrainer.trainer:__init__:68 - Found
earlystop_kwargs in settings.Set to None if you dont want earlystopping.
100%|         | 81/81 [00:00<00:00, 82.30it/s]
2025-09-29 11:13:36.369 | INFO          |
mltrainer.trainer:report:209 - Epoch 0 train
2.7253 test 2.3423 metric ['0.1359']
100%|         | 81/81 [00:00<00:00, 82.02it/s]
2025-09-29 11:13:37.482 | INFO          |
mltrainer.trainer:report:209 - Epoch 1 train
2.2043 test 2.1415 metric ['0.1984']
100%|         | 81/81 [00:01<00:00, 71.93it/s]
2025-09-29 11:13:38.715 | INFO          |
mltrainer.trainer:report:209 - Epoch 2 train
1.8576 test 1.6651 metric ['0.3297']
100%|         | 3/3 [00:03<00:00, 1.14s/it]

```

[57]: `trainer.loop()`

```
mlflow.end_run()
```

```

100%|         | 81/81 [00:00<00:00, 81.96it/s]
2025-09-29 11:13:39.827 | INFO          |
mltrainer.trainer:report:175 - Resuming epochs
from previous training at 3
2025-09-29 11:13:39.834 | INFO          |
mltrainer.trainer:report:209 - Epoch 3 train
1.4801 test 1.3199 metric ['0.4359']
100%|         | 81/81 [00:00<00:00, 82.03it/s]
2025-09-29 11:13:40.943 | INFO          |
mltrainer.trainer:report:209 - Epoch 4 train
1.2093 test 1.0398 metric ['0.5344']
100%|         | 81/81 [00:01<00:00, 66.71it/s]
2025-09-29 11:13:42.290 | INFO          |
mltrainer.trainer:report:209 - Epoch 5 train
1.0000 test 0.9280 metric ['0.5953']
100%|         | 3/3 [00:03<00:00, 1.18s/it]

```

we gaan hier conv1d toevoegen voor rnn

```
[58]: @dataclass
class HybridConfig:
    input_size: int          # aantal features (hier: 3 accelerometer-kanalen)
    hidden_size: int         # hidden size van RNN
    num_layers: int          # aantal RNN-lagen
    output_size: int         # aantal klassen (20 gestures)
    conv_channels: int = 16  # filters van Conv1D
    kernel_size: int = 3     # filtergrootte van Conv1D
    dropout: float = 0.0     # dropout voor regularisatie

[59]: class ConvLSTMmodel(nn.Module):
    def __init__(self, config: HybridConfig) -> None:
        super().__init__()
        self.config = config

        # Conv1D laag: zoekt korte patronen
        self.conv1 = nn.Conv1d(
            in_channels=config.input_size,          # input features (3
            ↪sensorkanalen)
            out_channels=config.conv_channels,      # aantal filters
            kernel_size=config.kernel_size
        )
        self.relu = nn.ReLU()

        # RNN laag (LSTM in dit voorbeeld)
        self.rnn = nn.LSTM(
            input_size=config.conv_channels,        # output van Conv1D = features
            ↪voor RNN
            hidden_size=config.hidden_size,
            num_layers=config.num_layers,
            dropout=config.dropout,
            batch_first=True,
        )

        # Lineaire laag naar klassen
        self.linear = nn.Linear(config.hidden_size, config.output_size)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (batch, timesteps, features) → (B, T, F)
        # Conv1D verwacht (batch, channels, timesteps)
        x = x.permute(0, 2, 1)  # (B, F, T)

        x = self.conv1(x)      # (B, conv_channels, T_new)
        x = self.relu(x)

        # terug naar (B, T_new, conv_channels) voor RNN
        x = x.permute(0, 2, 1)
```

```

x, _ = self.rnn(x)          # (B, T_new, hidden_size)
last_step = x[:, -1, :]    # laatste timestep
yhat = self.linear(last_step) # (B, output_size)
return yhat

```

```

[60]: config = HybridConfig(
    input_size=3,          # accelerometer x,y,z
    hidden_size=256,       # groter dan 64
    num_layers=2,          # begin met 1 LSTM-laag
    output_size=20,        # 20 gestures
    conv_channels=16,      # aantal conv filters
    kernel_size=3,         # kernelgrootte
    dropout=0.2,           # beetje dropout
)

```

```

[61]: with mlflow.start_run():
    mlflow.set_tag("model", "Conv1D-LSTM-16f-128h")
    mlflow.set_tag("dev", "nick")

    model = ConvLSTMmodel(config=config)

    trainer = Trainer(
        model=model,
        settings=settings,
        loss_fn=loss_fn,
        optimizer=optim.Adam,
        traindataloader=trainstreamer,
        validdataloader=validstreamer,
        scheduler=optim.lr_scheduler.ReduceLROnPlateau,
        device=device,
    )
    trainer.loop()

```

```

2025-09-29 11:13:42.346 | INFO      |
mltrainer.trainer:dir_add_timestamp:24 - Logging

```

```
to gestures/20250929-111342
```

```

2025-09-29 11:13:42.346 | INFO      |
mltrainer.trainer:__init__:68 - Found

```

```

earlystop_kwargs in settings.Set to None if you dont want earlystopping.
0%|          | 0/3 [00:00<?, ?it/s]2025-09-29

```

```

11:13:42.346 | INFO      |
mltrainer.trainer:__init__:68 - Found

```

```

earlystop_kwargs in settings.Set to None if you dont want earlystopping.
100%|         | 81/81 [00:01<00:00, 40.67it/s]

```

```

2025-09-29 11:13:44.584 | INFO      |
mltrainer.trainer:report:209 - Epoch 0 train
2.4787 test 2.1126 metric ['0.1953']
100%|      | 81/81 [00:01<00:00, 41.17it/s]
2025-09-29 11:13:46.760 | INFO      |
mltrainer.trainer:report:209 - Epoch 1 train
1.9419 test 1.8369 metric ['0.2500']
100%|      | 81/81 [00:02<00:00, 37.55it/s]
2025-09-29 11:13:49.128 | INFO      |
mltrainer.trainer:report:209 - Epoch 2 train
1.6494 test 1.5614 metric ['0.3672']
100%|      | 3/3 [00:06<00:00, 2.26s/it]

```

```
[62]: trainer.loop()
```

```

100%|      | 81/81 [00:02<00:00, 38.63it/s]
2025-09-29 11:13:51.449 | INFO      |
mltrainer.trainer:report:175 - Resuming epochs
from previous training at 3
2025-09-29 11:13:51.458 | INFO      |
mltrainer.trainer:report:209 - Epoch 3 train
1.3784 test 1.2981 metric ['0.4594']
100%|      | 81/81 [00:01<00:00, 42.77it/s]
2025-09-29 11:13:53.559 | INFO      |
mltrainer.trainer:report:209 - Epoch 4 train
1.2531 test 1.1349 metric ['0.5062']
100%|      | 81/81 [00:01<00:00, 43.95it/s]
2025-09-29 11:13:55.620 | INFO      |
mltrainer.trainer:report:209 - Epoch 5 train
1.0657 test 0.9477 metric ['0.6031']
100%|      | 3/3 [00:06<00:00, 2.16s/it]

```

```
[63]: trainer.loop()
```

```

100%|      | 81/81 [00:02<00:00, 39.06it/s]
2025-09-29 11:13:57.902 | INFO      |
mltrainer.trainer:report:175 - Resuming epochs
from previous training at 6
2025-09-29 11:13:57.908 | INFO      |
mltrainer.trainer:report:209 - Epoch 6 train
0.8970 test 0.7593 metric ['0.6922']
100%|      | 81/81 [00:01<00:00, 44.70it/s]
2025-09-29 11:13:59.917 | INFO      |

```

```
mltrainer.trainer:report:209 - Epoch 7 train
0.6815 test 0.5740 metric ['0.7844']
100%|      | 81/81 [00:01<00:00, 44.35it/s]
2025-09-29 11:14:01.952 | INFO      |
mltrainer.trainer:report:209 - Epoch 8 train
0.4688 test 0.4253 metric ['0.8516']
100%|      | 3/3 [00:06<00:00, 2.11s/it]
```

```
[64]: trainer.loop()
```

```
100%|      | 81/81 [00:02<00:00, 39.51it/s]
2025-09-29 11:14:04.188 | INFO      |
mltrainer.trainer:report:175 - Resuming epochs
from previous training at 9
2025-09-29 11:14:04.194 | INFO      |
mltrainer.trainer:report:209 - Epoch 9 train
0.3338 test 0.2389 metric ['0.9437']
100%|      | 81/81 [00:01<00:00, 43.70it/s]
2025-09-29 11:14:06.250 | INFO      |
mltrainer.trainer:report:209 - Epoch 10 train
0.2226 test 0.1819 metric ['0.9453']
100%|      | 81/81 [00:01<00:00, 43.01it/s]
2025-09-29 11:14:08.341 | INFO      |
mltrainer.trainer:report:209 - Epoch 11 train
0.1545 test 0.3434 metric ['0.9109']
2025-09-29 11:14:08.341 | INFO      |
mltrainer.trainer:__call__:252 - best loss:
0.1819, current loss 0.3434.Counter 1/5.
100%|      | 3/3 [00:06<00:00, 2.13s/it]
```

```
[65]: trainer.loop()
```

```
100%|      | 81/81 [00:02<00:00, 37.91it/s]
2025-09-29 11:14:10.682 | INFO      |
mltrainer.trainer:report:175 - Resuming epochs
from previous training at 12
2025-09-29 11:14:10.689 | INFO      |
mltrainer.trainer:report:209 - Epoch 12 train
0.1947 test 0.0745 metric ['0.9797']
100%|      | 81/81 [00:01<00:00, 43.09it/s]
2025-09-29 11:14:12.775 | INFO      |
mltrainer.trainer:report:209 - Epoch 13 train
0.0712 test 0.0547 metric ['0.9891']
```

```

100%|      | 81/81 [00:01<00:00, 43.20it/s]
2025-09-29 11:14:14.850 | INFO      |
mltrainer.trainer:report:209 - Epoch 14 train
0.0937 test 0.0760 metric ['0.9812']
2025-09-29 11:14:14.850 | INFO      |
mltrainer.trainer:__call__:252 - best loss:
0.0547, current loss 0.0760.Counter 1/5.
100%|      | 3/3 [00:06<00:00, 2.17s/it]

```

[66]: `trainer.loop()`

```

100%|      | 81/81 [00:02<00:00, 39.52it/s]
2025-09-29 11:14:17.110 | INFO      |
mltrainer.trainer:report:175 - Resuming epochs
from previous training at 15
2025-09-29 11:14:17.117 | INFO      |
mltrainer.trainer:report:209 - Epoch 15 train
0.0792 test 0.0777 metric ['0.9766']
2025-09-29 11:14:17.117 | INFO      |
mltrainer.trainer:__call__:252 - best loss:
0.0547, current loss 0.0777.Counter 2/5.
100%|      | 81/81 [00:01<00:00, 44.54it/s]
2025-09-29 11:14:19.130 | INFO      |
mltrainer.trainer:report:209 - Epoch 16 train
0.0438 test 0.0624 metric ['0.9781']
2025-09-29 11:14:19.130 | INFO      |
mltrainer.trainer:__call__:252 - best loss:
0.0547, current loss 0.0624.Counter 3/5.
100%|      | 81/81 [00:01<00:00, 42.57it/s]
2025-09-29 11:14:21.235 | INFO      |
mltrainer.trainer:report:209 - Epoch 17 train
0.0373 test 0.0409 metric ['0.9859']
100%|      | 3/3 [00:06<00:00, 2.13s/it]

```

[67]: `trainer.loop()`

```

100%|      | 81/81 [00:02<00:00, 40.40it/s]
2025-09-29 11:14:23.442 | INFO      |
mltrainer.trainer:report:175 - Resuming epochs
from previous training at 18
2025-09-29 11:14:23.449 | INFO      |
mltrainer.trainer:report:209 - Epoch 18 train
0.0277 test 0.0114 metric ['0.9984']

```

```

100%|      | 81/81 [00:01<00:00, 42.33it/s]
2025-09-29 11:14:25.563 | INFO      |
mltrainer.trainer:report:209 - Epoch 19 train

0.0401 test 0.0819 metric ['0.9766']
2025-09-29 11:14:25.564 | INFO      |
mltrainer.trainer:__call__:252 - best loss:

0.0114, current loss 0.0819.Counter 1/5.
100%|      | 81/81 [00:01<00:00, 43.99it/s]
2025-09-29 11:14:27.609 | INFO      |
mltrainer.trainer:report:209 - Epoch 20 train

0.0228 test 0.0243 metric ['0.9938']
2025-09-29 11:14:27.610 | INFO      |
mltrainer.trainer:__call__:252 - best loss:

0.0114, current loss 0.0243.Counter 2/5.
100%|      | 3/3 [00:06<00:00, 2.12s/it]

```

[68]: `trainer.loop()`

```

100%|      | 81/81 [00:02<00:00, 40.15it/s]
2025-09-29 11:14:29.852 | INFO      |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 21
2025-09-29 11:14:29.859 | INFO      |
mltrainer.trainer:report:209 - Epoch 21 train

0.0440 test 0.0230 metric ['0.9938']
2025-09-29 11:14:29.859 | INFO      |
mltrainer.trainer:__call__:252 - best loss:

0.0114, current loss 0.0230.Counter 3/5.
100%|      | 81/81 [00:01<00:00, 44.01it/s]
2025-09-29 11:14:31.897 | INFO      |
mltrainer.trainer:report:209 - Epoch 22 train

0.0160 test 0.0216 metric ['0.9938']
2025-09-29 11:14:31.898 | INFO      |
mltrainer.trainer:__call__:252 - best loss:

0.0114, current loss 0.0216.Counter 4/5.
100%|      | 81/81 [00:01<00:00, 42.49it/s]
2025-09-29 11:14:34.022 | INFO      |
mltrainer.trainer:report:209 - Epoch 23 train

0.0079 test 0.0105 metric ['0.9984']
100%|      | 3/3 [00:06<00:00, 2.14s/it]

```

[69]: `trainer.loop()`

```

100%|      | 81/81 [00:02<00:00, 39.92it/s]
2025-09-29 11:14:36.256 | INFO      |
mltrainer.trainer:report:175 - Resuming epochs
from previous training at 24
2025-09-29 11:14:36.263 | INFO      |
mltrainer.trainer:report:209 - Epoch 24 train
0.0092 test 0.0132 metric ['0.9969']
2025-09-29 11:14:36.263 | INFO      |
mltrainer.trainer:__call__:252 - best loss:
0.0105, current loss 0.0132.Counter 1/5.
100%|      | 81/81 [00:01<00:00, 44.78it/s]
2025-09-29 11:14:38.277 | INFO      |
mltrainer.trainer:report:209 - Epoch 25 train
0.0075 test 0.0064 metric ['0.9984']
100%|      | 81/81 [00:01<00:00, 43.73it/s]
2025-09-29 11:14:40.324 | INFO      |
mltrainer.trainer:report:209 - Epoch 26 train
0.0037 test 0.0077 metric ['0.9984']
2025-09-29 11:14:40.325 | INFO      |
mltrainer.trainer:__call__:252 - best loss:
0.0064, current loss 0.0077.Counter 1/5.
100%|      | 3/3 [00:06<00:00, 2.10s/it]

```

```
[70]: mlflow.end_run()
```